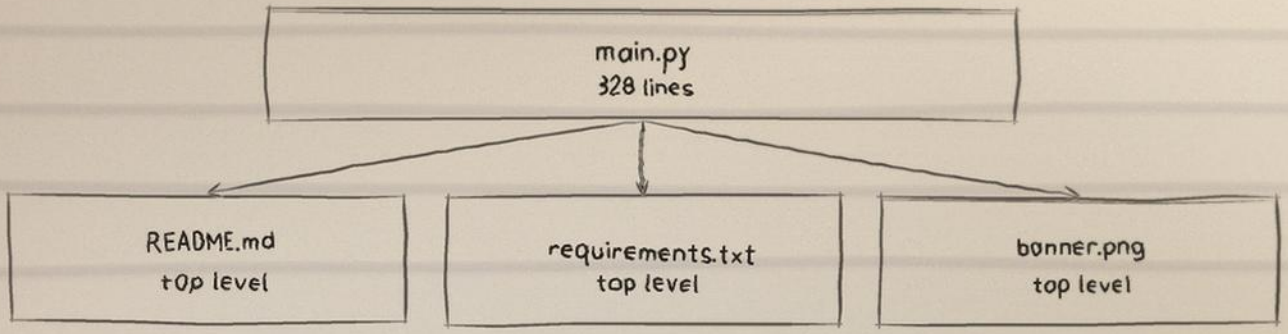
Dep Vuln Dependencylicensechecker: how it is put together



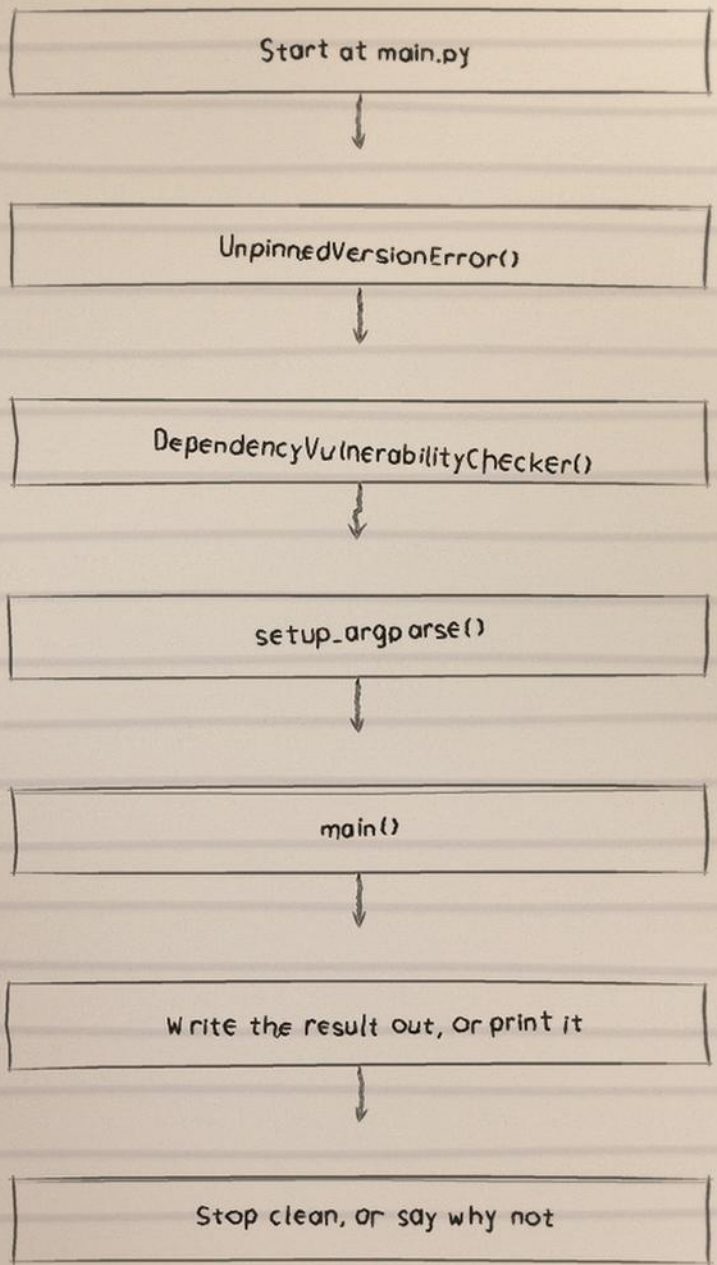
the names doing the work

UnpinnedVersionError, DependencyVulnerabilityChecker, setup_argparse
main, __init__, load_dependencies

Note on versions: dependency entries are stored as requirement specifiers. The OSV check needs one concrete version, so only a pinned entry (a single == version, or a bare version.

sketched from the files, CK

Dep Vuln Dependencylicensechecker: the run, step by st



If a step fails, the honest run says so and stops.

traced, not guessed, OK

Product map: what Dep Vuln Dependencylicensechecker is for

WHAT IT IS

Note on versions: dependency entries are stored as requirement specifiers. The OSV check needs one concrete version, so only a pinned entry (a single == version, or a bare version.

WHAT GOES IN

The files at the top of the tree: main.py, README.md, requirements.txt.

WHAT COMES OUT

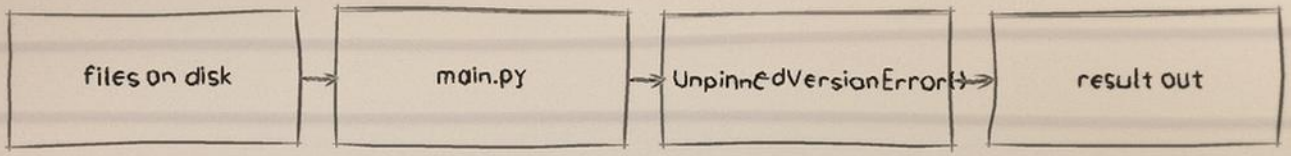
What the top files produce when they run or get read. Nothing else is promised.

WHERE IT STOPS

No service behind it, no account, no upload. If a path is untested, VISUAL.md says so.

mapped by hand,

Dep Vuln Dependencylicensechecker: data flow, in one direction

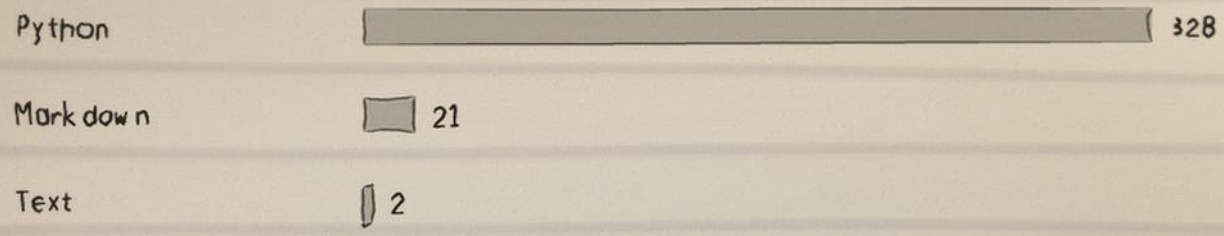


One direction. Anything the code does not do is not drawn.

second sheet, GK

Dep Vuln Dependencylicensechecker in numbers, counted 2026-10-30

Counted from the local snapshot of this repo. Method for every number: VISUAL.md.



5 files in the tree.
8 commits, first 2025-04-15, last 2026-10-30, from the local git log.
NO test files by name in this snapshot.

counted and drawn, GK

Dep Vuln Dependencylicensechecker: schematic of the apparatus

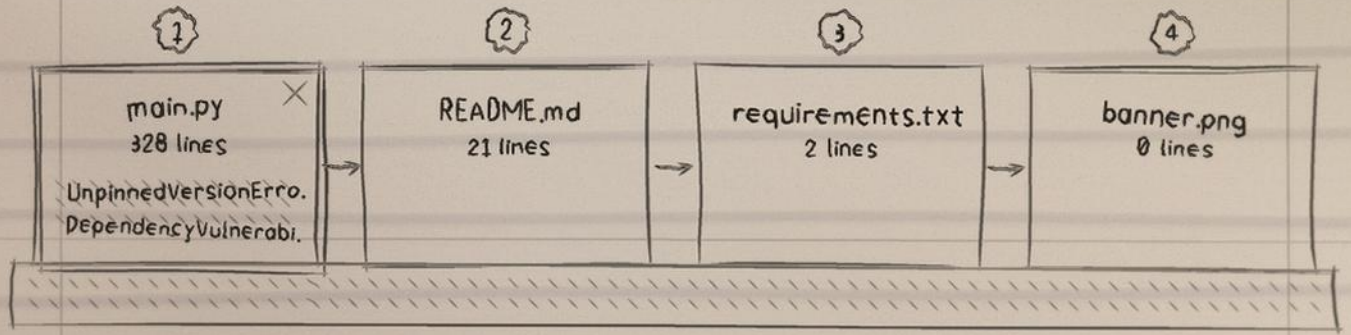
the system as a machine: parts, flow, and measurements

construction lines left in, as drawn

parts list

1. main.py
2. README.md
3. requirements.txt
4. banner.png

in flow order, first to last



base plate: the repo itself, sectioned

span: 5 files, 351 lines counted

turns of the crank so far: 8 commits, 2025-04-15 to 2026-10-30

measured and sketched, GK